DAVAO DEL NORTE STATE COLLEGE
Institute of Computing - Bachelor of Science in Information Technology

CampusMart POS

Campus Convenience Store Point-of-Sale System

Scenario 5 - IT 415 Application Development and Emerging Technologies
Midterm Practical Examination, First Semester, SY 2026-2027

Student	TODO: Student Name
Year / Section	TODO: Year / Section
Date	October 13, 2026
GitHub Repository	https://github.com/Ajjime/Scenario-5-Campus-Convenience-Store-POS

1. Overview

CampusMart is a point-of-sale web application for a small campus convenience store selling snacks, drinks, and basic school supplies. A cashier browses products, builds a cart, enters a cash payment, and receives a confirmation and a digital receipt. All data is kept in memory; no database, login, or inventory is used, as allowed by Section VII of the exam. It runs in a desktop browser with mouse and keyboard.

2. Tech Stack and How to Run

Tool	Purpose
React 19	UI components and state
Vite	Dev server and build tool
Tailwind CSS 4	Styling
oxlint	Linting
Git / GitHub	Version control

Install and run: `npm install` then `npm run dev`, and open http://localhost:5173.

3. Product List

Product	Price
Instant Noodles	₱18.00
Potato Chips	₱25.00
Soft Drink (Can)	₱25.00
Bottled Water	₱20.00
Ballpen (piece)	₱12.00
Notebook (piece)	₱35.00

4. Transaction Flow (Exam Section IV)

No.	Stage	Implementation	Code
1	Browse Products	Grid with photo, name, price; search and category filter	ProductList.jsx
2	Add to Cart	Add to Cart button; + / - quantity buttons	App.jsx, Cart.jsx
3	Review Order	Items, quantities, subtotals, total	OrderSummary.jsx
4	Enter Payment	Numeric cash amount field	PaymentForm.jsx
5	Validate and Compute	Checks the amount; computes change	validatePayment.js
6	Confirm Payment	Confirmation with transaction number, date, time	PaymentConfirmation.jsx
7	Receipt / New Transaction	Digital receipt; New Transaction resets all	Receipt.jsx, App.jsx

5. Required Features Checklist (Exam Section V)

#	Requirement	Status	Implementation
1	Display at least 5 products with names and prices	Done (6)	products array, ProductList
2	Add products to cart via on-screen buttons	Done	addToCart
3	Increase / decrease quantity	Done	increaseQuantity, decreaseQuantity
4	Remove a product from the cart	Done	removeFromCart
5	Auto-compute subtotals and total	Done	total recomputed each render
6	Clear order summary of cart items	Done	OrderSummary
7	Accept cash through a numeric input	Done	PaymentForm
8	Validate blank, non-numeric, negative, insufficient	Done	validatePayment
9	Change = Amount Paid - Total	Done	validatePayment (centavo math)
10	Payment confirmation	Done	PaymentConfirmation
11	Digital receipt with transaction reference	Done	Receipt, createTransactionNumber
12	Start new transaction, clear all data	Done	startNewTransaction

6. Design Notes

• **State lives in App.jsx**: cart, completedSale (a snapshot taken at payment, or null), a form key, and a short success message. Child components receive data and callbacks as props.
• **The total is derived**, not stored: total = sum of price x quantity, so it is never out of date.
• **Money math uses whole centavos** (Math.round(x * 100)) to avoid floating-point errors such as 0.1 + 0.2.
• **After payment the sale is locked.** The snapshot cannot change, and the cart and payment form are disabled until New Transaction.
• **Transaction number** format is TXN-YYYYMMDD-HHMMSS, taken from the system clock.
• **New Transaction** clears the cart and the receipt, and remounts the payment form by changing its React key, which clears the amount and errors.

Validation rules

Input	Result
Cart is empty	"Your cart is empty. Please add at least one item before paying."
Blank, non-numeric, negative, more than 2 decimals, or above ₱1,000,000	"Please enter a valid payment amount."
Less than the total	"Insufficient payment. Please enter at least ₱X.XX."
Equal to or more than the total	Accepted; change = paid - total

7. Application Screenshots

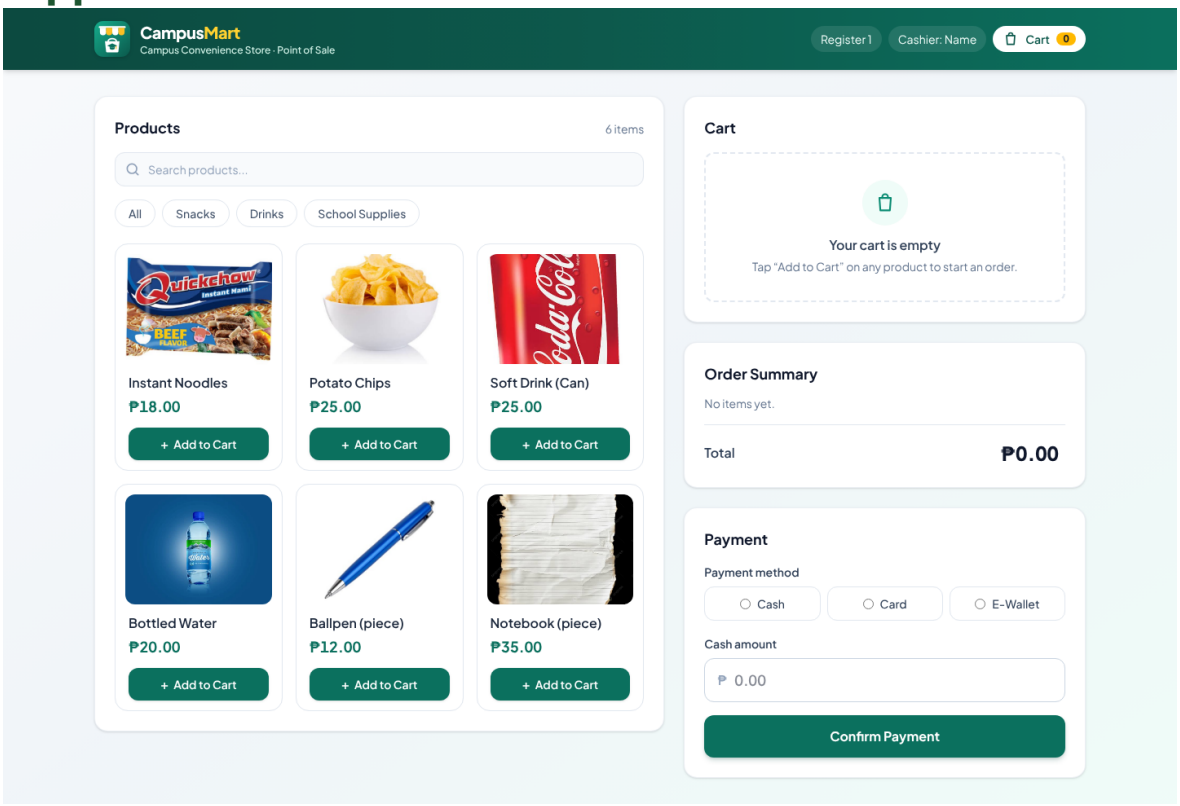


Figure 1. Product list (step 1: browse products)

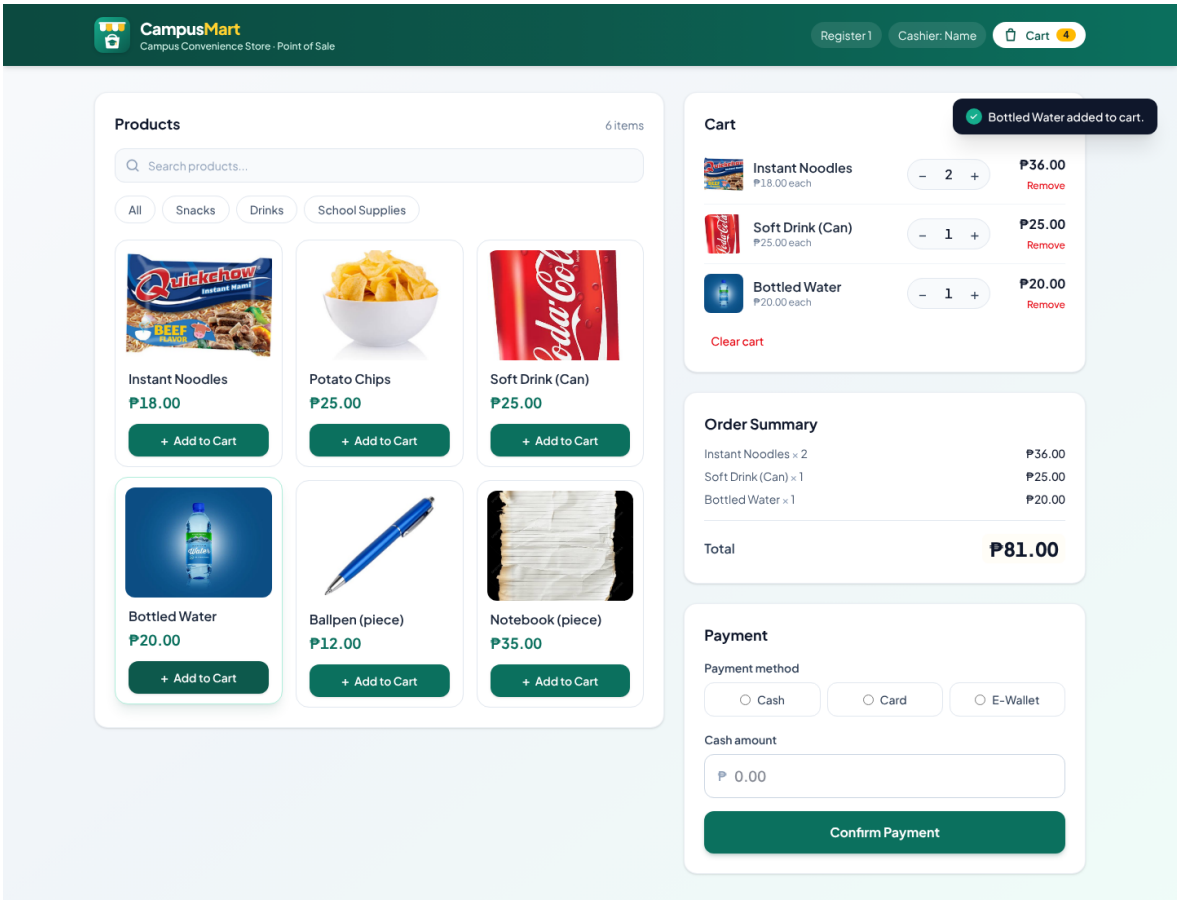


Figure 2. Cart and order summary after adding items (steps 2-3)

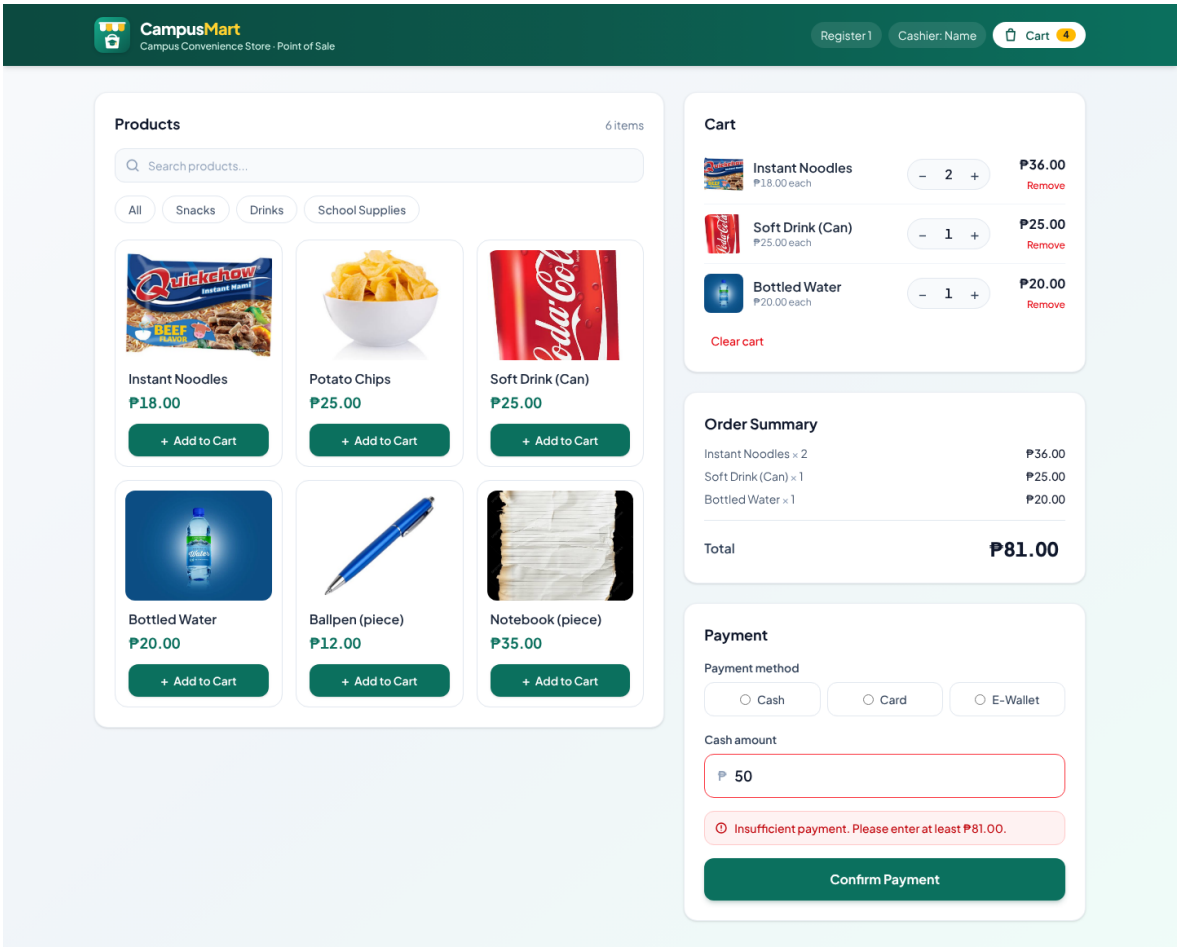


Figure 3. Payment screen showing the insufficient-payment validation message (steps 4-5)

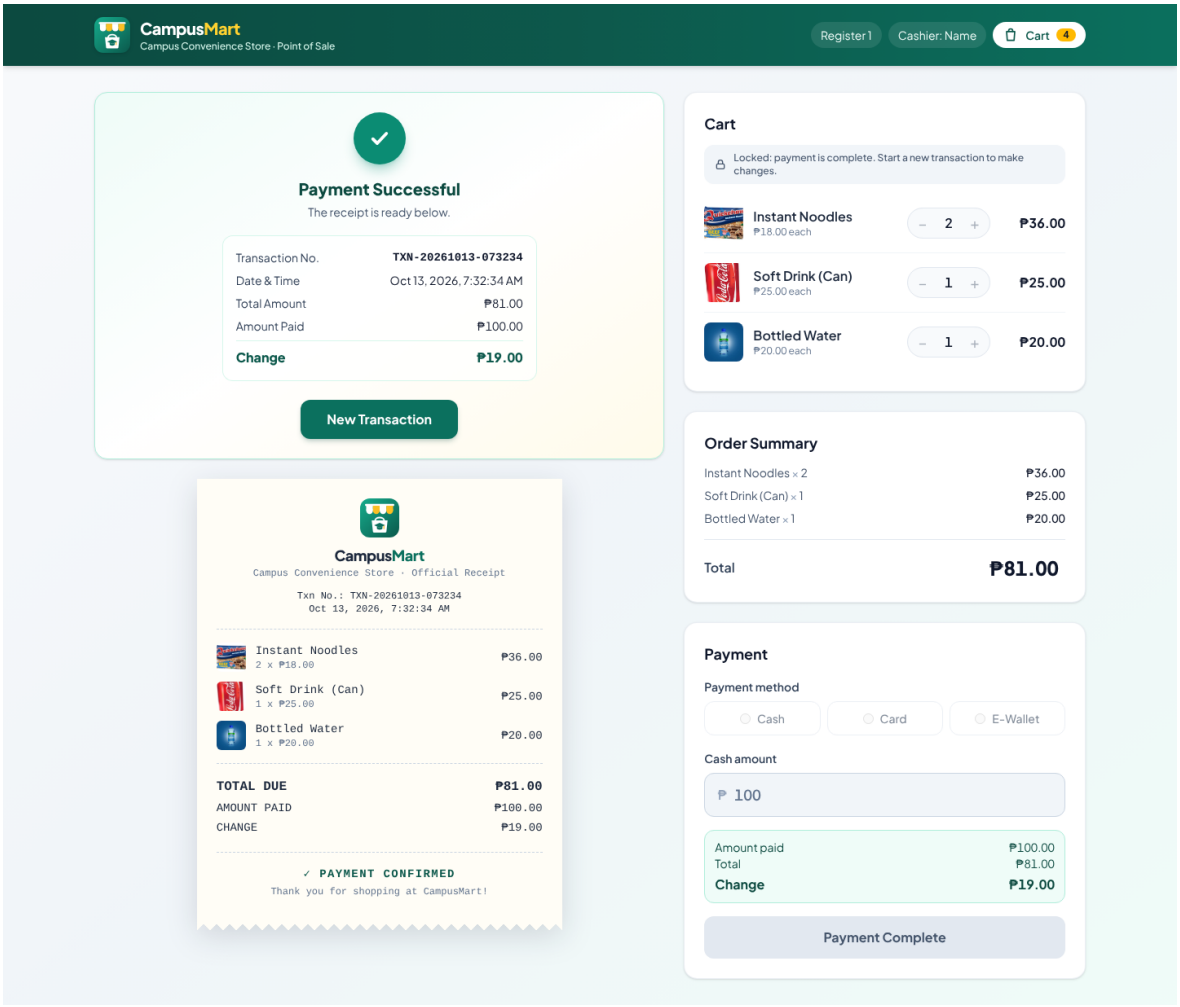


Figure 4. Payment confirmation with transaction number and change (step 6)

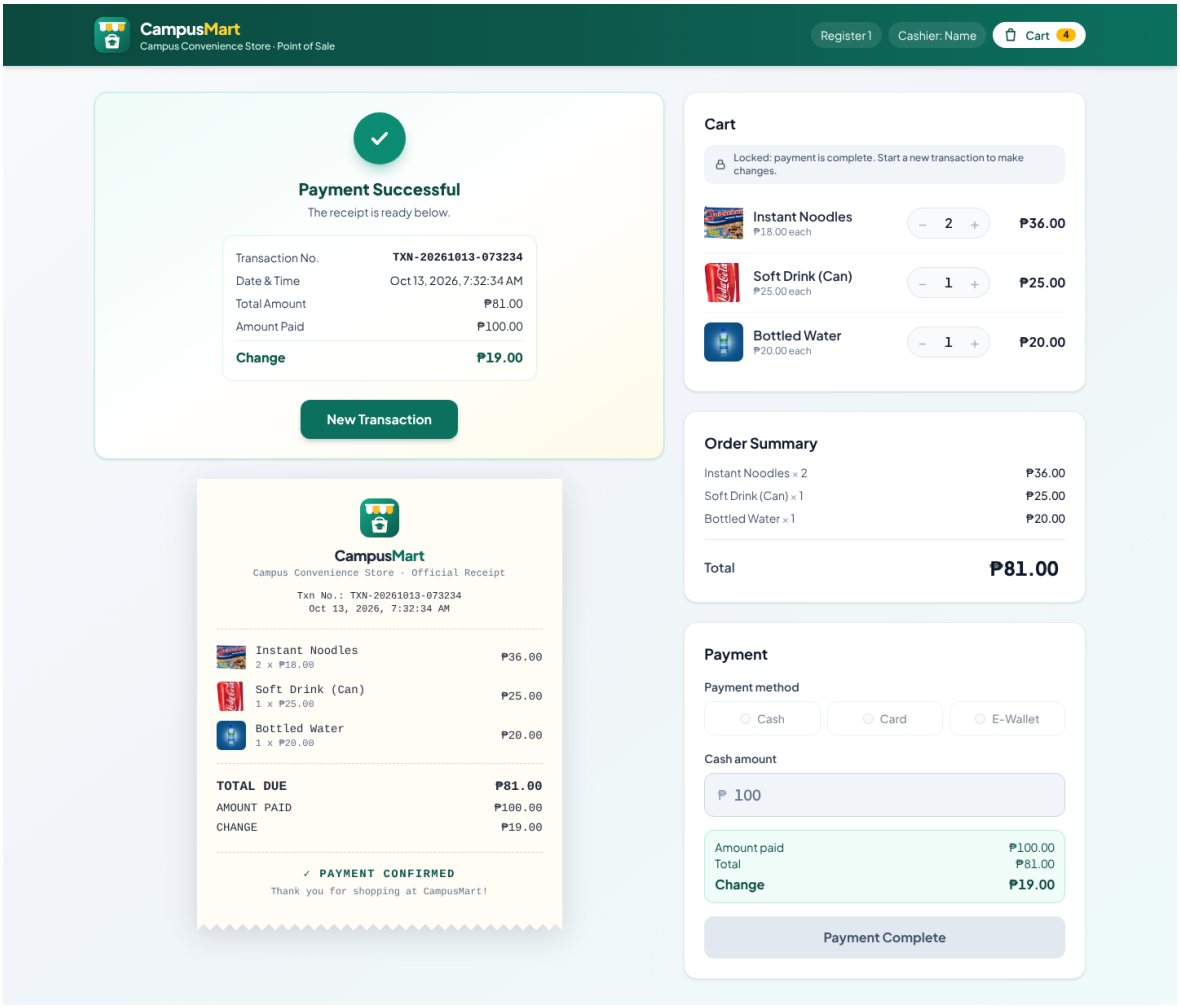


Figure 5. Digital receipt with items, totals, amount paid, change, and transaction number (step 7)

8. Test Cases

Validation results were produced by running validatePayment directly with a total of ₱165.00 (11 of 11 passed).

#	Input	Expected	Result
1	(blank)	Invalid-amount message	Pass
2	abc	Invalid-amount message	Pass
3	-50	Invalid-amount message	Pass
4	100	Insufficient payment, at least ₱165.00	Pass
5	164.99	Insufficient payment	Pass
6	165	Accepted, change ₱0.00	Pass
7	200	Accepted, change ₱35.00	Pass
8	200 (with spaces)	Accepted, change ₱35.00	Pass
9	20.555	Invalid-amount message	Pass
10	1e3	Invalid-amount message	Pass
11	Any amount with empty cart	Empty-cart message	Pass

Worked example

2 Instant Noodles (₱36.00) + 1 Soft Drink (₱25.00) + 1 Bottled Water (₱20.00) = total ₱81.00. Paid ₱100.00, change ₱19.00. This is exactly what Figures 3 to 5 show.

9. Version Control (Exam Section IX)

Commit	Message	Purpose
6c31464	Initial commit: Set up project and basic interface	Project setup
9f09e33	Add product list and basic user interface	Interface
cd10e2c	Implement cart and order calculations	Core ordering and computation
98946cb	Add payment validation and digital receipt	Validation and receipt
be8e569	Add product photos, logo, and UI animations	Improvements
fcc3ba2	Fix bugs and improve application	Debugging
07de899	Add README and credits	Documentation

GitHub commit history screenshot: **TODO - to be inserted**.

10. AI-Assisted Development (Exam Section VIII)

AI tool used: **TODO**

Prompt 1 - Building a feature

Field	Entry
Prompt used	TODO
Purpose of the prompt	TODO
AI response (summary)	TODO
Your evaluation	TODO
Modifications you made	TODO

Prompt 2 - Debugging or refactoring

Field	Entry
Prompt used	TODO
Purpose of the prompt	TODO
AI response (summary)	TODO
Your evaluation	TODO
Modifications you made	TODO

11. Credits

Product photos: **TODO (source and photographer per README)**. The CampusMart logo is an original design made for this project.